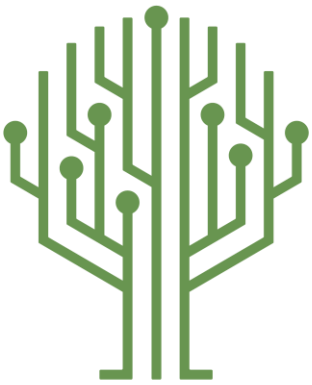


Green Pace Secure Development Policy

Contents

Overview 2
Purpose 2
Scope 2
Module Three Milestone 2
Ten Core Security Principles 2
C/C++ Ten Coding Standards 3
Coding Standard 1 4
Coding Standard 2 5
Coding Standard 3 6
Coding Standard 4 7
Coding Standard 5 8
Coding Standard 6 9
Coding Standard 7 10
Coding Standard 8 11
Coding Standard 9 13
Coding Standard 10 14
Defense-in-Depth Illustration 15
Project One 15
1. Revise the C/C++ Standards 15
2. Risk Assessment 15
3. Automated Detection 15
4. Automation 15
5. Summary of Risk Assessments 16
6. Create Policies for Encryption and Triple A 16
7. Map the Principles 17
Audit Controls and Management 18
Enforcement 18
Exceptions Process 18
Distribution 19
Policy Change Control 19
Policy Version History 19
Appendix A Lookups 19
Approved C/C++ Language Acronyms 19

Overview

Software development at Green Pace requires consistent implementation of secure principles to all developed applications. Consistent approaches and methodologies must be maintained through all policies that are uniformly defined, implemented, governed, and maintained over time.

Purpose

This policy defines the core security principles; C/C++ coding standards; authorization, authentication, and auditing standards; and data encryption standards. This article explains the differences between policy, standards, principles, and practices (guidelines and procedure): Understanding the Hierarchy of Principles, Policies, Standards, Procedures, and Guidelines.

Scope

This document applies to all staff that create, deploy, or support custom software at Green Pace.

Module Three Milestone

Ten Core Security Principles

Principles	Write a short paragraph explaining each of the 10 principles of security.
1. Validate Input Data	Ensure that all untrusted input data is validated against expected lengths, formats, and types. Validating input data prevents buffer overflow, common injections, and cross cite scripting
2. Heed Compiler Warnings	Compiler warnings should be heeded and fixed as warnings can show bugs, memory leaks, or undefined behaviors that can be exploited if not fixed.
3. Architect and Design for Security Policies	Integrating security requirements into the structure of the program rather than an external security system ensures that the system would be resilient.
4. Keep It Simple	Reducing redundancy in the program ensures that testing the program for vulnerabilities and errors is easier and reduces the amount of potential failure points in the system.
5. Default Deny	Access to the system should be denied by default unless explicitly permitted. Systems should utilize whitelist approaches to ensure that anyone not explicitly allowed to access the system cannot access the system
6. Adhere to the Principle of Least Privilege	Limiting users to the minimum access right required to complete designated tasks ensures that compromised users would do minimal damage.
7. Sanitize Data Sent to Other Systems	Ensure that all data passed to external systems are encoded or cleansed, filtered, and properly escaped. This is best to prevent backend attacks
8. Practice Defense in Depth	Implement multiple layers of redundant security measures to ensure that if one layer fails, the other layers of security prevent full exploitation of the vulnerability.
9. Use Effective	Utilize quality assurance methodologies throughout the development cycle, such as static code testing, dynamic testing, automated unit testing, and peer code reviews

Principles	Write a short paragraph explaining each of the 10 principles of security.
Quality Assurance Techniques	
10. Adopt a Secure Coding Standard	Adhere to well defined secure coding practices and standards based on the coding environment used; this ensures that security quality is consistent.

C/C++ Ten Coding Standards

Complete the coding standards portion of the template according to the Module Three milestone requirements. In Project One, follow the instructions to add a layer of security to the existing coding standards. Please start each standard on a new page, as they may take up more than one page. The first seven coding standards are labeled by category. The last three are blank so you may choose three additional standards. Be sure to label them by category and give them a sequential number for that category. Add compliant and noncompliant sections as needed to each coding standard.

Green Pace

Coding Standard 1

Coding Standard	Label	Name of Standard
Data Type	STD-001-CPP	Obey the one-definition rule

Noncompliant Code

Two translations for a class with the same name but differing definitions

```
// a.cpp
struct S {
  int a;
};

// b.cpp
class S {
public:
  int a;
};
```

Compliant Code

Use a header file to introduce the object to both translations

```
// S.h
struct S {
  int a;
};

// a.cpp
#include "S.h"

// b.cpp
#include "S.h"
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 4 and 10: Consistent definition of classes prevents memory layout mismatches, structural ambiguity, and undefined behavior

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Unlikely	Medium	6	2

Automation

Tool	Version	Checker	Description Tool
Astrée	25.10	**class-inconsistent-definitions** **definition-duplicate** **external-file-spreading** **type-compatibility** **type-file-spreading** **undefined-extern** **undefined-extern-pure-virtual**	Partially checked
CodeSonar	27.0	**LANG.STRUCT.DEF.FDH** **LANG.STRUCT.DEF.ODH**	Function defined in header file Object defined in header file
LDRA tool suite	9.7.1	**286 S, 287 S**	Fully implemented
Parasoft C/C++test	2026.1	**CERT_CPP-DCL60-a**	The One Definition Rule shall not be violated

Coding Standard 2

Coding Standard	Label	Name of Standard
Data Value	STD-002-CPP	Do not access an object outside of its lifetime

Noncompliant Code

A pointer to an object is used to call a non-static member function of the object prior to the beginning of the pointer's lifetime

```
struct S {
  void mem_fn();
};

void f() {
  S *s;
  s->mem_fn();
}
```

Compliant Code

Storage is obtained for the pointer prior to calling S::mem_fn()

```
struct S {
  void mem_fn();
};

void f() {
  S *s = new S;
  s->mem_fn();
  delete s;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 2 and 8: Prevents uninitialized pointer dereferencing and execution of arbitrary code via dangling references

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Probable	Medium	6	2

Automation

Green Pace

Tool	Version	Checker	Description Tool
Astrée	25.10	**csa-stack-address-escape** **return-reference-local** **dangling_pointer_use**	Partially checked
Clang	3.9	**-Wdangling-initializer-list**	Catches some lifetime issues related to incorrect use of std::initializer_list<>
CodeSonar	27.0	**IO.UAC** **ALLOC.UAF**	Use after close Use after free
LDRA tool suite	9.7.1	**42 D, 53 D, 77 D, 1 J, 71 S, 565 S**	Partially implemented

Green Pace

Coding Standard 3

Coding Standard	Label	Name of Standard
String Correctness	STD-003-CPP	Range check element access

Noncompliant Code

the value returned by the call to get_index() may be greater than the number of elements stored in the string, resulting in undefined behavior

```
#include <string>


extern std::size_t get_index();


void f() {

  std::string s("01234567");

  s[get_index()] = '1';

}
```

Compliant Code

uses the std::basic_string::at() function, which behaves in a similar fashion to the index operator[] but throws a std::out_of_range exception if pos >= size()

```
## Compliant Solution (Range Check)

This compliant solution checks that the value returned by `get_index()` is
within a valid range before calling `operator[]().`

::code-block{quality="good"}

``` cpp
#include <string>

extern std::size_t get_index();

void f() {
```
```

Compliant Code

```
  std::string s("01234567");
  std::size_t i = get_index();
  if (i < s.length()) {
    s[i] = '1';
  } else {
    // Handle error
  }
}
  }
  /* ... */
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 1 and 8: Prevents buffer overflows and out of bounds memory writes

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Unlikely	Low	3	3

Automation

Tool	Version	Checker	Description Tool
Astrée	25.10	**assert_failure**	
CodeSonar	27.0	**LANG.MEM.BO** **LANG.MEM.BU** **LANG.MEM.TBA** **LANG.MEM.TO** **LANG.MEM.TU**	Buffer overrun Buffer underrun Tainted buffer access Type overrun Type underrun
Parasoft C/C++test	2026.1	**CERT_CPP-STR53-a**	Guarantee that container indices are within the valid range
Polyspace Bug Finder	R2025b	CERT C++: STR53-CPP	Checks for: • Array access out of bounds • Array access with tainted index • Pointer dereference with tainted offset Rule partially covered.

Green Pace

Coding Standard 4

Coding Standard	Label	Name of Standard
SQL Injection	STD-004-CPP	Prevent SQL injection

Noncompliant Code

incorporating the unsanitized input argument username into the SQL command, allowing an attacker to inject validuser' OR '1'='1

```
sql = "SELECT * FROM Users WHERE Name = " + username + " AND Pass = " + password + "
```

Compliant Code

Preventing SQL injection by sanitizing and validating inputs

```
sql = "SELECT * FROM Users WHERE Name = %s AND Pass = %s;",
username,userpassword};
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 1 and 7: Guarantees that user parameters are kept distinct from SQL logic

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Likely	Medium	18	1

Automation

Tool	Version	Checker	Description Tool
The Checker Framework	2.1.3	**Tainting Checker**	Trust and security errors (see Chapter 8)
Parasoft Jtest	2026.1	**CERT.IDS00.TDSQL**	Protect against SQL injection
Security Reviewer - Static Reviewer	6.02	**Injection10**	Full implementation
SonarQube	9.9	**S2077** **S3649**	Executing SQL queries is security-sensitive SQL queries should not be vulnerable to injection attacks

Coding Standard 5

Coding Standard	Label	Name of Standard
Memory Protection	STD-005-CPP	Do not access freed memory

Noncompliant Code

s is dereferenced after it has been deallocated. If this access results in a write-after-free, the vulnerability can be exploited to run arbitrary code with the permissions of the vulnerable process.

```
#include <new>

struct S {
  void f();
};

void g() noexcept(false) {
  S *s = new S;
  // ...
  delete s;
  // ...
  s->f();
}
```

Compliant Code

The dynamically allocated memory is not deallocated until it is no longer required

```
#include <new>

struct S {
  void f();
};

void g() noexcept(false) {
  S *s = new S;
  // ...
  s->f();
  delete s;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 2 and 4: Utilizing smart pointers eliminates manual memory deallocation errors

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Likely	Low	9	2

Automation

Tool	Version	Checker	Description Tool
Astrée	25.10	**csa-double-free** **csa-use-after-free** **dangling_pointer_use**	partially checked + soundly supported
Clang	3.9	clang-analyzer-cplusplus.NewDelete clang-analyzer-alpha.security.ArrayBoundV2	Checked by clang-tidy , but does not catch all violations of this rule.
CodeSonar	27.0	**ALLOC.UAF**	Use after free
Coverity	v7.5.0	**USE_AFTER_FREE**	Can detect the specific instances where memory is deallocated more than once or read/written to the target of a freed pointer

Coding Standard 6

Coding Standard	Label	Name of Standard
Assertions	STD-006-CPP	Incorporate diagnostic tests using assertions

Noncompliant Code

The assert() macro to assert a property concerning a memory-mapped structure that is essential for the code to behave correctly

```
#include <assert.h>

struct timer {
  unsigned char MODE;
  unsigned int DATA;
  unsigned int COUNT;
};

int func(void) {
  assert(sizeof(struct timer) == sizeof(unsigned char) + sizeof(unsigned int) +
sizeof(unsigned int));
}
```

Compliant Code

For assertions involving only constant expressions, a preprocessor conditional statement may be used

```
struct timer {
  unsigned char MODE;
  unsigned int DATA;
  unsigned int COUNT;
};

#if (sizeof(struct timer) != (sizeof(unsigned char) + sizeof(unsigned int) +
sizeof(unsigned int)))
  #error "Structure must not have any padding"
#endif
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 2 and 9: Converts runtime assertion failures into compile time checks, ensuring binary structural integrity

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Low	Unlikely	Medium	1	3

Automation

Tool	Version	Checker	Description Tool
CodeSonar	27.0	**LANG.FUNCS.ASSERTS**	Not enough assertions
Coverity	2017.07	**ASSERT_SIDE_EFFECT**	Can detect the specific instance where assertion contains an operation/function call that may have a side effect
Parasoft C/C++test	2026.1	**CERT_C-MSC11-a**	Assert liberally to document internal assumptions and invariants
Security Reviewer - Static Reviewer	6.02	**CPPPBE**	Fully implemented

Coding Standard 7

Coding Standard	Label	Name of Standard
Exceptions	STD-007-CPP	Handle all exceptions

Noncompliant Code

Neither f() nor main() catch exceptions thrown by throwing_func()

```
void throwing_func() noexcept(false);

void f() {
  throwing_func();
}

int main() {
  f();
}
```

Compliant Code

The main entry point handles all exceptions, which ensures that the stack is unwound up to the main() function and allows for graceful management of external resources

```
void throwing_func() noexcept(false);

void f() {
  throwing_func();
}

int main() {
  try {
    f();
  } catch (...) {
    // Handle error
  }
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 3 and 8: Ensures state preservation, resource cleanup and application stability during unexpected failures

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Low	Probable	Medium	6	2

Automation

Tool	Version	Checker	Description Tool
Astrée	25.10	**early-catch-all** **exception-propagation-outside-catch-all** **main-function-catch-all** **unhandled-throw-noexcept**	Partially checked
CodeSonar	27.0	**LANG.STRUCT.UCTCH** **PARSE.MBDH**	Masked by handler Masked by default handler
Parasoft C/C++test	2026.1	**CERT_CPP-ERR51-a** **CERT_CPP-ERR51-b**	Always catch exceptions Each exception explicitly thrown in the code shall have a handler of a compatible type in all call paths that could lead to that point
Polyspace Bug Finder	R2025b	CERT C++: ERR51-CPP	Checks for unhandled exceptions (rule partially covered)

Coding Standard 8

Coding Standard	Label	Name of Standard
Input Output	STD-008-CPP	Reset strings on fgets() or fgetws() failure

Noncompliant Code

An error flag is set if fgets() fails. However, buf is not reset and has indeterminate contents

```
#include <stdio.h>   enum { BUFFER_SIZE = 1024 }; void func(FILE *file) {
char buf[BUFFER_SIZE];

if (fgets(buf, sizeof(buf), file) == NULL) { /* Set error flag and
continue */ } }
```

Compliant Code

buf is set to an empty string if fgets() fails. The equivalent solution for fgetws() would set buf to an empty wide string

```
#include <stdio.h>   enum { BUFFER_SIZE = 1024 };

void func(FILE *file) { char buf[BUFFER_SIZE];

if (fgets(buf, sizeof(buf), file) == NULL) { /* Set error flag and
continue */ *buf = '\0'; } }
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 1 and 5: Prevents stale memory or uninitialized buffers from being passes to downstream processing

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Low	Probable	Medium	6	2

Automation

Tool	Version	Checker	Description Tool
CodeSonar	27.0	**LANG.MEM.UVAR**	Uninitialized Variable

Tool	Version	Checker	Description Tool
LDRA tool suite	10.6.0	**44 S**	Implemented and enhanced
Parasoft C/C++test	2026.1	**CERT_C-FIO40-a**	Reset strings on fgets() or fgetws() failure
Polyspace Bug Finder	R2025b	CERT C: Rule FIO40-C	Checks for use of indeterminate string (rule partially covered)

Coding Standard 9

Coding Standard	Label	Name of Standard
Containers	STD-009-CPP	Use Valid iterator ranges

Noncompliant Code

Incrementing the iterator representing the past-the-end element of the range results in undefined behavior

```
#include <algorithm>
#include <iostream>
#include <vector>

void f(const std::vector<int> &c) {
  std::for_each(c.end(), c.begin(), [](int i) { std::cout << i; });
}
```

Compliant Code

The iterator values passed to std::for_each() are passed in the proper order

```
#include <algorithm>
#include <iostream>
#include <vector>

void f(const std::vector<int> &c) {
  std::for_each(c.begin(), c.end(), [](int i) { std::cout << i; });
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 4 and 10: Guarantees correct iterator boundaries, preventing runtime segmentation faults and memory corruption

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Probable	High	6	2

Automation

Tool	Version	Checker	Description Tool
Astrée	25.10	**overflow_upon_dereference**	
CodeSonar	27.0	**LANG.MEM.BO**	Buffer Overrun
Parasoft C/C++test	2026.1	**CERT_CPP-CTR53-a** **CERT_CPP-CTR53-b**	Do not use an iterator range that isn't really a range Do not compare iterators from different containers
Polyspace Bug Finder	R2025b	CERT C++: CTR53-CPP	Checks for invalid iterator range (rule partially covered).

Coding Standard 10

Coding Standard	Label	Name of Standard
Object Oriented Programming	STD-010-CPP	Do not use pointer-to-member operators to access nonexistent members

Noncompliant Code

When called on an object whose dynamic type is D , the pointer-to-member call is well defined. However, the dynamic type of the underlying object is B , which results in undefined behavior

```
struct B {
  virtual ~B() = default;
};

struct D : B {
  virtual ~D() = default;
  virtual void g() { /* ... */ }
};

void f() {
  B *b = new B;

  // ...

  void (B::*gptr)() = static_cast<void(B::*)()>(&D::g);
  (b->*gptr)();
  delete b;
}
```

Compliant Code

The upcast is removed, rendering the initial code ill-formed and emphasizing the underlying problem that B::g() does not exist

```
struct B {
  virtual ~B() = default;
};

struct D : B {
  virtual ~D() = default;
  virtual void g() { /* ... */ }
};

void f() {
  B *b = new D; // Corrected the dynamic object type.
```

Compliant Code

```
  // ...
  void (D::*gptr)() = &D::g; // Moved static_cast to the next line.
  (static_cast<D *>(b)->*gptr)();
  delete b;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): 3 and 10: Avoids invalid vtable dispatch and arbitrary code execution through type mismatches

Threat Level

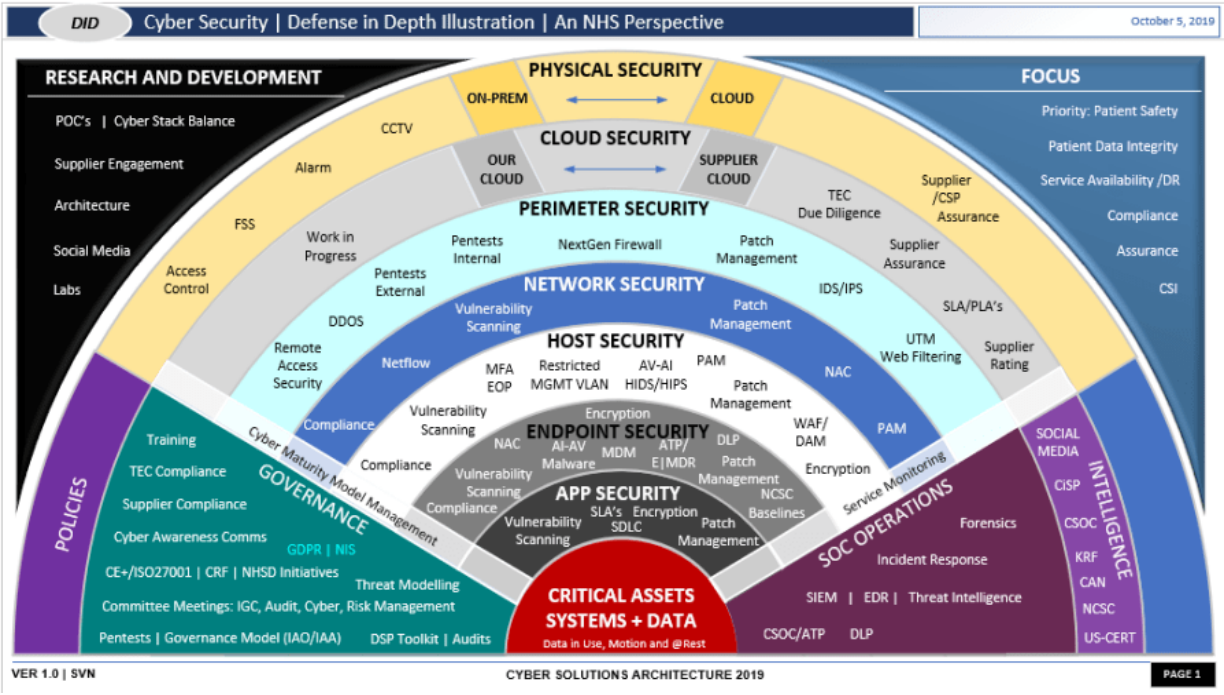
Severity	Likelihood	Remediation Cost	Priority	Level
High	Probable	High	6	2

Automation

Tool	Version	Checker	Description Tool
CodeSonar	27.0	**LANG.MEM.UVAR**	Uninitialized Variable
Parasoft C/C++test	2026.1	**CERT_CPP-OOP55-a**	A cast shall not convert a pointer to a function to any other pointer type, including a pointer to function type
Parasoft Insure++			Runtime detection
Polyspace Bug Finder	R2025b	CERT C++: OOP55-CPP	Checks for pointers to member accessing non-existent class members (rule fully covered).

Defense-in-Depth Illustration

This illustration provides a visual representation of the defense-in-depth best practice of layered security.



Project One

There are seven steps outlined below that align with the elements you will be graded on in the accompanying rubric. When you complete these steps, you will have finished the security policy.

Revise the C/C++ Standards

You completed one of these tables for each of your standards in the Module Three milestone. In Project One, add revisions to improve the explanation and examples as needed. Add rows to accommodate additional examples of compliant and noncompliant code. Coding standards begin on the security policy.

Risk Assessment

Complete this section on the coding standards tables. Enter high, medium, or low for each of the headers, then rate it overall using a scale from 1 to 5, 5 being the greatest threat. You will address each of the seven policy standards. Fill in the columns of severity, likelihood, remediation cost, priority, and level using the values provided in the appendix.

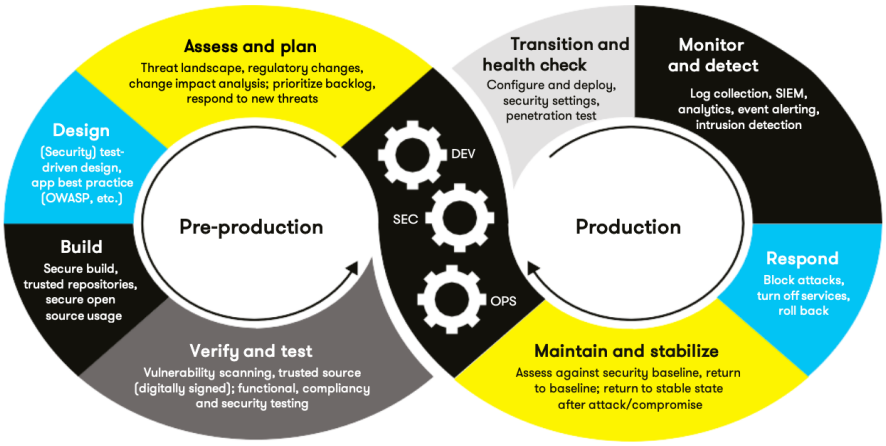
Automated Detection

Complete this section of each table on the coding standards to show the tools that may be used to detect issues. Provide the tool name, version, checker, and description. List one or more tools that can automatically detect this issue and its version number, name of the rule or check (preferably with link), and any relevant comments or description—if any. This table ties to a specific C++ coding standard.

Automation

Provide a written explanation using the image provided.

Green Pace



Automation will be used for the enforcement of and compliance to the standards defined in this policy. Green Pace already has a well-established DevOps process and infrastructure. Define guidance on where and how to modify the existing DevOps process to automate enforcement of the standards in this policy. Use the DevSecOps diagram and provide an explanation using that diagram as context.

[Insert your written explanations here.]

Summary of Risk Assessments

Consolidate all risk assessments into one table including both coding and systems standards, ordered by standard number.

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-001-CPP	High	Unlikely	Medium	6	2
STD-002-CPP	High	Probable	Medium	6	2
STD-003-CPP	High	Unlikely	Low	3	3
STD-004-CPP	High	Likely	Medium	18	1
STD-005-CPP	High	Likely	Low	9	2
STD-006-CPP	Low	Unlikely	Medium	1	3
STD-007-CPP	Low	Probable	Medium	6	2
STD-008-CPP	Low	Probable	Medium	6	2
STD-009-CPP	High	Probable	High	6	2
STD-010-CPP	High	Probable	High	6	2

Create Policies for Encryption and Triple A

Include all three types of encryption (in flight, at rest, and in use) and each of the three elements of the Triple-A framework using the tables provided**.**

a. Explain each type of encryption, how it is used, and why and when the policy applies.
b. Explain each type of Triple-A framework strategy, how it is used, and why and when the policy applies.

Write policies for each and explain what it is, how it should be applied in practice, and why it should be used.

a. Encryption	**Explain what it is and how and why the policy applies.**
Encryption at rest	All sensitive, stored data must be encrypted using AES-256. This policy applies to prevent unauthorized physical or logical data extraction from storage media.
Encryption in flight	All network traffic carrying systems or user data must be encrypted using TLS 1.3. Legacy protocols are prohibited to protect data against sniffing and man-in-the-middle attacks
Encryption in use	Process memory containing sensitive secrets must be stored in secure, non-pageable memory segments and scrubbed immediately after execution

b. Triple-A Framework*	**Explain what it is and how and why the policy applies.**
Authentication	Multi-Factor Authentication is required for all user and administrative access. All authentication attempts must be logged with timestamps and client addresses
Authorization	Role-Based Access Control and the Principle of Least Privilege must govern user entitlements. Access checks must occur server-side before serving any requested resource
Accounting	Comprehensive, immutable audit logs must record critical platform events, including user logins, access level changes, database modifications, user management actions, and file accesses

*****Use this checklist for the Triple A to be sure you include these elements in your policy:

- User logins
- Changes to the database
- Addition of new users
- User level of access
- Files accessed by users

Map the Principles

Map the principles to each of the standards, and provide a justification for the connection between the two. In the Module Three milestone, you added definitions for each of the 10 principles provided. Now it's time to connect the standards to principles to show how they are supported by principles. You may have more than one principle for each standard, and the principles may be used more than once. Principles are numbered 1 through 10. You will list the number or numbers that apply to each standard, then explain how each of these principles supports the standard. This exercise demonstrates that you have based your security policy on widely accepted principles. Linking principles to standards is a best practice.

NOTE: Green Pace has already successfully implemented the following:

- Operating system logs
- Firewall logs
- Anti-malware logs

The only item you must complete beyond this point is the Policy Version History table.

Audit Controls and Management

Every software development effort must be able to provide evidence of compliance for each software deployed into any Green Pace managed environment.

Evidence will include the following:

- Code compliance to standards
- Well-documented access-control strategies, with sampled evidence of compliance
- Well-documented data-control standards defining the expected security posture of data at rest, in flight, and in use
- Historical evidence of sustained practice (emails, logs, audits, meeting notes)

Enforcement

The office of the chief information security officer (OCISO) will enforce awareness and compliance of this policy, producing reports for the risk management committee (RMC) to review monthly. Every system deployed in any environment operated by Green Pace is expected to be in compliance with this policy at all times.

Staff members, consultants, or employees found in violation of this policy will be subject to disciplinary action, up to and including termination.

Exceptions Process

Any exception to the standards in this policy must be requested in writing with the following information:

- Business or technical rationale
- Risk impact analysis
- Risk mitigation analysis
- Plan to come into compliance
- Date for when the plan to come into compliance will be completed

Approval for any exception must be granted by chief information officer (CIO) and the chief information security officer (CISO) or their appointed delegates of officer level.

Exceptions will remain on file with the office of the CISO, which will administer and govern compliance.

Distribution

This policy is to be distributed to all Green Pace IT staff annually. All IT staff will need to certify acceptance and awareness of this policy annually.

Policy Change Control

This policy will be automatically reviewed annually, no later than 365 days from the last revision date. Further, it will be reviewed in response to regulatory or compliance changes, and on demand as determined by the OCISO.

Policy Version History

Version	Date	Description	Edited By	Approved By
1.0	08/05/2020	Initial Template	David Buksbaum	
1.1	09/14/2026	Milestone	Brian Har	[Insert text.]
1.2	10/05/2026	Project 1	Brian Har	[Insert text.]

Appendix A Lookups

Approved C/C++ Language Acronyms

Language	Acronym
C++	CPP
C	CLG
Java	JAV